



UNIVERSIDAD
TECNOLÓGICA
METROPOLITANA
del Estado de Chile

Informe Grupal

Fundamentos de Computación Paralela, Concurrencia, Diseño Algorítmico y OpenMP

Asignatura: Computación Paralela y Distribuida

Sección: 412

Fecha de entrega: 28 de mayo, 2026

Profesor: Michael Miranda Sandoval

Estudiante: Christian Pérez

Benjamín Zamorano

1.- Resumen Ejecutivo:

El presente informe técnico expone el diseño, la implementación y la evaluación experimental de estrategias de computación paralela aplicadas a tres problemas críticos de la Ciencia de Datos. El objetivo central es optimizar el uso de la infraestructura de hardware multinúcleo para mitigar los cuellos de botella temporales y espaciales asociados al procesamiento de grandes volúmenes de información.

A través de un enfoque metodológico, se abordaron tres escenarios con distintas demandas arquitectónicas:

1. **Ejercicio 1 (Normalización Masiva en C++/OpenMP):** Se implementó un algoritmo *Memory-Bound* para estructurar, limpiar y estandarizar una matriz sintética de $50.000.000 \times 16$ observaciones con un 5% de valores nulos (NaN). Mediante una estrategia de particionamiento estático por filas y la aplicación de reducciones locales concurrentes en tres fases síncronas, se eliminó la falsa compartición (*false sharing*). Los resultados funcionales validaron el correcto aislamiento de los datos válidos y la ausencia matemática de valores atípicos ($|z| > 3$) debido a la naturaleza de la distribución base, logrando reducir el tiempo de cómputo secuencial y exponiendo el límite de escalabilidad lineal impuesto por la Pared de la Memoria (Memory Wall) al incrementar el número de hilos..
2. **Ejercicio 2 (Pipeline de Logs en Python/Multiprocessing):** Se diseñó una arquitectura distribuida basada en el paradigma de paso de mensajes por bloques (*streaming chunks*) para procesar un archivo masivo comprimido (.log.gz) y consolidar métricas de rendimiento en ventanas temporales de 15 minutos. El diseño evadió las restricciones del bloqueo global del intérprete (GIL) mediante el uso de ProcessPoolExecutor. La evaluación evidenció que, a diferencia de C++, el rendimiento en Python se ve fuertemente condicionado por el *overhead* de serialización de objetos (pickle) e intercomunicación de procesos (IPC), introduciendo una degradación en la eficiencia que ilustra los límites prácticos de la Ley de Amdahl en flujos I/O-bound. El pipeline terminó con la persistencia física de los datos agrupados en el archivo estructurado final.
3. **Ejercicio 3 (Similitud por Bloques para Embeddings en C++/OpenMP):** Se resolvió un problema de alta complejidad temporal ($O(n^2 \cdot d)$) para encontrar los 10 vecinos más cercanos de 20.000 vectores de dimensión 128. Respetando la prohibición estricta de almacenar la matriz de similitudes en memoria, se diseñó un esquema *CPU-Bound* que procesó la matriz triangular simétrica de forma *in-place*. Al asociar un montículo máximo privado por hilo

(std::priority_queue), el consumo de memoria RAM se redujo drásticamente de 3.2 GB a menos de 25 MB, permitiendo que las estructuras críticas residieran por completo en la memoria caché L3. La planificación dinámica (schedule(dynamic, 64)) resolvió eficientemente el desbalance de carga latente, logrando una aceleración experimental de 4.98x con 8 hilos y garantizando una precisión exacta (Recall@10 = 1.0).

Los benchmarks obtenidos en las tres experiencias demuestran que la optimización de algoritmos en ciencia de datos no depende únicamente del incremento de unidades de procesamiento, sino de la comprensión rigurosa de la jerarquía de memoria del hardware, la minimización de la sincronización costosa y la correcta selección de los mecanismos de paralelización según la naturaleza del problema.

2.- Ejercicio 1: Normalización Masiva en C++/OpenMP

2.1. Diagnóstico del Problema y Restricciones Arquitectónicas

El problema exige el procesamiento y transformación estandarizada de una matriz masiva X de dimensiones $n \times d$, donde $n = 50.000.000$ filas, y $d = 16$ columnas. El volumen de datos bruto impacta directamente la arquitectura de hardware a través de las siguientes limitantes:

- **Restricción de Memoria Espacial:** Cada elemento de la matriz se almacena en precisión doble (double, 8 bytes). La matriz de entrada X requiere un espacio en la memoria RAM de $50.000.000 \times 16 \times 8 \text{ bytes} \approx 6,4 \text{ GB}$. Dado que se solicita construir y almacenar de forma simultánea una matriz normalizada Z de idéntico tamaño, el consumo base del algoritmo se eleva a **12,8 GB**. Esto clasifica la carga como un problema estrictamente enfocado en los datos (*Data-Intensive*), al borde de las capacidades de memoria en estaciones de trabajo comerciales estándar.
- **Dependencia de Datos Lineal:** Existe una restricción de orden secuencial en las operaciones matemáticas. No es posible computar la matriz normalizada Z ni evaluar el criterio de valores atípicos ($|z| > 3$) de forma directa. El algoritmo depende de que primero se consolide la media aritmética de cada columna (ignorando los valores NaN), para luego calcular la varianza muestral de las mismas. Una vez obtenidos ambos vectores estadísticos escalares, recién se puede iniciar la fase de estandarización.

- **Tratamiento de Valores Faltantes:** La presencia de elementos NaN (5% del conjunto de datos) introduce la necesidad de un control de flujo condicional iterativo (*if (!isnan(val))*) para evitar la propagación de valores indeterminados en las sumas y raíces cuadradas, lo cual altera sutilmente el balance dinámico entre hilos.

2.2. Contraste Crítico de las Propuestas Iniciales y Lecciones Aprendidas

El diseño de nuestra solución final no nació de una única idea, sino de un debate técnico en el que analizamos los aciertos y errores de nuestras propuestas individuales. Al contrastar las dos miradas, pudimos rescatar lo mejor de cada una y corregir ciertos vacíos conceptuales.

Por un lado, nuestra primera propuesta identificó que necesitábamos un mecanismo de reducción concurrente para los acumuladores, anticipando correctamente que variables globales como el conteo de atípicos y los valores válidos iban a provocar condiciones de carrera si no se controlaban. Sin embargo, al plasmarlo en el papel hubo un par de tropiezos técnicos: se sugirió la cláusula *reduction(*)*, operador que sirve para multiplicaciones secuenciales y no para sumas, donde corresponde usar *+*, y se atribuyó erróneamente a la "localidad de memoria" el control de las carreras de datos, confundiendo la optimización de las cachés con las herramientas de sincronización arquitectónica.

Por otro lado, la segunda propuesta aportó una visión estructural al plantear un fraccionamiento del trabajo por filas mediante un *parallel for*. Esta decisión es importante para el compilador de C/C++ porque, al almacenar las matrices de forma contigua en memoria (*row-major order*), recorrerlas horizontalmente maximiza la localidad espacial de la caché. Además, se propuso la elección de *schedule(static)*, ya que la carga por fila es prácticamente homogénea y no justifica pagar el costo de un planificador dinámico. No obstante, propuso de forma ambigua la apertura y cierre constante del bloque paralelo. Esto último habría sido desastroso en la práctica, introduciendo un *overhead* severo por la creación y destrucción redundante de hilos en el sistema operativo.

Al final, la discusión grupal nos permitió entender que no bastaba con paralelizar el código a ciegas. La solución óptima requería fusionar la estructura limpia del recorrido por filas con una estrategia de privatización absoluta que evitara el cuello de botella de las regiones críticas dentro de los bucles masivos.

2.2.1. Solución final y justificación:

Para estructurar la solución definitiva, decidimos descartar la idea de abrir y cerrar bloques paralelos en cada paso del proceso, optando en su lugar por mantener una única región paralela activa (`#pragma omp parallel`) que englobara todo el ciclo de vida del algoritmo. Al hacer esto, eliminamos de golpe el *overhead* que le significa al sistema operativo crear, sincronizar y destruir hilos tres veces consecutivas para la media, la varianza y la normalización final.

El corazón de nuestro diseño se basa en la privatización absoluta de los datos intermedios para neutralizar el fenómeno de la falsa compartición (*false sharing*). Si hubiésemos configurado un vector global para que todos los hilos sumaran sus filas directamente sobre las 16 columnas en tiempo real, los diferentes núcleos de la CPU habrían pasado más tiempo invalidándose mutuamente sus líneas de caché que procesando operaciones matemáticas. Para evitar este cuello de botella, le entregamos a cada hilo un pequeño set de buffers locales privados (`suma_local` y `validos_local`) de apenas 16 celdas. Cada hilo procesa el bloque de filas que le corresponde de manera totalmente aislada y a la máxima velocidad que le permite su hardware.

El recorrido interno de la matriz se diseñó respetando el almacenamiento por filas de C/C++ (*row-major order*). En lugar de saltar entre columnas, el bucle principal recorre de manera horizontal la memoria física con una planificación estática (`#pragma omp for schedule(static)`). Dado que cada fila de la matriz tiene exactamente el mismo tamaño (16 columnas) y el descarte de los valores faltantes (NaN) mediante el filtro condicional `isnan()` toma un tiempo imperceptible en la CPU, la carga de trabajo por bloque es homogénea. La planificación estática nos garantiza que OpenMP distribuya los bloques equitativamente al inicio de la ejecución, reduciendo a cero el costo de administración dinámica en un bucle masivo de 50 millones de iteraciones.

La única zona de sincronización explícita del código ocurre al final de cada fase, cuando los hilos terminan de barrer sus respectivas porciones de la matriz. En ese instante, y solo una vez por hilo, ingresamos a una directiva `#pragma omp critical` para volcar los acumuladores locales privados en los vectores globales que consolidan la media y la varianza. Inmediatamente después, una barrera síncrona (`#pragma omp barrier`) detiene temporalmente los hilos para asegurar que las sumas estén completas, permitiendo que un único hilo (`#pragma omp single`) compute las divisiones finales de los promedios y las desviaciones estándar antes de continuar con la siguiente etapa del pipeline.

2.3. Análisis Crítico

¿Por qué *critical* dentro del bucle masivo es una mala decisión?

Colocar la directiva `#pragma omp critical` dentro del cuerpo interno del bucle paralelo (por ejemplo, al evaluar cada una de las 50.000.000 de filas) representa una falla de diseño crítica que destruye por completo el paralelismo.

La directiva *critical* obliga al sistema operativo y a la arquitectura de hardware a serializar la ejecución mediante el uso de cerrojos (*locks*) mutuos. Si los hilos deben adquirir y liberar un cerrojo único $50.000.000 \times 16 = 800.000.000$ de veces, el costo transaccional administrativo del hardware (*overhead* de sincronización) supera por varios órdenes de magnitud al tiempo de cómputo matemático neto. Como consecuencia, el algoritmo paralelo se vuelve drásticamente más lento que una ejecución secuencial simple, provocando una contención masiva de los núcleos por el acceso a la sección bloqueada.

Análisis de Error Numérico en Punto Flotante por Orden de Reducción

En la arquitectura computacional moderna, la aritmética de punto flotante no es asociativa. Esto significa matemáticamente que:

$$(a + b) + c \neq a + (b + c)$$

Cuando un bucle de acumulación masiva de variables reales (double o float) se ejecuta de manera secuencial, el procesador suma los elementos en un orden estrictamente lineal ($X_0 + X_1 + X_2 + \dots$). Sin embargo, al paralelizar con OpenMP utilizando reducción o buffers locales, la matriz se fragmenta y el orden en que se agrupan y suman los valores parciales cambia radicalmente según la velocidad y finalización de cada hilo en los diferentes núcleos de la CPU.

Dado que la representación binaria en punto flotante genera pequeños errores de redondeo al sumar o comparar números que tienen órdenes de magnitud muy distantes, el resultado numérico final de la media y la varianza presentará sutiles variaciones en sus dígitos decimales flotantes al variar el número de hilos ($p=1,2,4,8$). Este fenómeno es propio de la computación científica de alto rendimiento y no constituye un error de código, sino una limitación física de la representación binaria.

2.4. Pruebas Experimentales y Resultados de Benchmark

Los experimentos se ejecutaron bajo un entorno controlado y automatizado utilizando el compilador nativo de la plataforma de trabajo, realizando repeticiones tras un ciclo inicial de calentamiento (*warm-up*) de la caché del sistema.

Tabla de Rendimiento e Infraestructura Paralela

Cantidad de Hilos (p)	Tiempo de Ejecución (T_p)[s]	Speedup Obtenido (S_p)	Eficiencia Muestral (E_p)
1 (secuencial)	42,66436s	1,00000x	100,00000%
2	25,32250s	1,68484x	84,24198%
4	11,73257s	3,63640x	90,91006%
8	7,80522s	5,46613x	68,32664%

2.5. Interpretación de Métricas y Escalabilidad

Los datos experimentales revelan un escalamiento positivo y altamente competitivo, logrando reducir el tiempo de procesamiento desde los 42,66 segundos en su versión secuencial hasta un mínimo de apenas 7,80 segundos al exprimir los 8 hilos de la infraestructura. El comportamiento del *Speedup* es sobresaliente en la primera mitad del espectro, alcanzando 1,68x con 2 hilos y tocando un máximo de 3,63x con 4 hilos. Esto empuja la eficiencia de los cuatro núcleos a un excelente 90,91%, lo que demuestra que la decisión de privatizar los acumuladores locales en buffers de 16 celdas logró mantener el procesamiento casi por completo dentro de la memoria caché del procesador, manteniendo a raya el *false sharing*.

Sin embargo, al dar el salto hacia los 8 hilos, la curva de rendimiento abandona la tendencia lineal y el *Speedup* se estabiliza en 5,46x, provocando que la eficiencia disminuya al 68,32%. Este quiebre en el rendimiento marca el punto exacto donde la naturaleza del problema (estrictamente *Memory-Bound*) pasa la cuenta. Al tener 8 hilos lógicos exigiendo de forma simultánea la lectura y escritura de celdas de memoria para procesar operaciones matemáticas tan directas (sumas flotantes planas), los canales físicos de la placa madre colapsan. El sistema choca finalmente contra la Pared de la Memoria (*Memory Wall*), donde la CPU se ve obligada a

introducir ciclos de espera inactivos (*stall cycles*) a la expectativa de que el bus de datos complete las transferencias con la RAM principal, validando así el límite de escalabilidad que predice la Ley de Amdahl.

2.6. Tabla de Resultados Estadísticos Finales (Por Columna)

El pipeline volcó la consolidación funcional de la matriz masiva ignorando adecuadamente los datos nulos:

Columna	Valores Válidos Detectados	Media (Ignorando NaN)	Varianza (Muestral)	Atípicos ($ z > 3$)
Columna 1	47497678	54,9971	674,9321	0
Columna 2	47499002	54,9983	674,9433	0
...	...	...	...	...
Columna 15	47499501	54,9975	674,9061	0
Columna 16	47499930	54,9968	675,0339	0

Análisis Crítico de Atípicos:

La detección sistemática de exactamente 0 valores atípicos en todas las columnas valida rigurosamente la consistencia matemática de la simulación. Al utilizar una función sintética basada en una distribución uniforme continua, los datos quedan estrictamente acotados dentro de un rango predefinido. Teóricamente, la desviación estándar de una distribución uniforme equivale a $\sigma = \frac{b-a}{\sqrt{12}}$, lo que restringe el valor máximo posible de una puntuación Z a $\sqrt{3} \approx 1,732$. Al ser matemáticamente imposible que un elemento supere la cota de $|z| > 3$, los resultados confirman la precisión exacta de la lógica algorítmica desarrollada.

3.- Ejercicio 2: Pipeline concurrente de procesamiento de logs en Python

3.1. Diagnóstico del Pipeline y Clasificación de Cargas

El procesamiento analítico de un volumen de 2 GB de archivos de registro comprimidos (.log.gz) impone un desafío de diseño debido a la naturaleza mixta de sus operaciones. Para estructurar una estrategia eficiente, el flujo de trabajo se fragmentó en tres etapas consecutivas, clasificando su demanda de recursos de la siguiente manera:

- **Etap 1: Lectura física del archivo (Estrictamente I/O-bound):** La transferencia de los bloques de bytes desde el almacenamiento secundario hacia la memoria RAM está limitada por la tasa de transferencia de lectura del disco duro y los llamados de sistema operativos.
- **Etap 2: Descompresión y Parseo de Cadenas (Estrictamente CPU-bound):** Contrario a la intuición matemática inicial, la descompresión algorítmica del formato Gzip y el posterior desglose de cada línea de texto, aislando mediante división de cadenas las variables de user_id, timestamp, endpoint, latencia y código, son tareas de alta demanda computacional que saturan las unidades aritméticas del procesador.
- **Etap 3: Agregación en Ventanas Temporales (CPU-bound):** La conversión de las marcas de tiempo a objetos cronológicos para agrupar las métricas de latencia promedio y conteos en bloques discretos de 15 minutos opera sobre estructuras de datos dinámicas en memoria, consumiendo ciclos netos de procesamiento.

3.2. Contraste Crítico de las Propuestas Iniciales y Lecciones Aprendidas

La arquitectura final de nuestro pipeline es el resultado de un análisis crítico de nuestras propuestas individuales, lo que nos permitió corregir imprecisiones teóricas y consolidar un diseño híbrido robusto.

En nuestra primera aproximación al problema, tuvimos el acierto de identificar que el parseo y la limpieza de las líneas de log debían ejecutarse mediante procesos independientes para evadir las restricciones del entorno de Python. Sin embargo, cometimos el error técnico de catalogar la descompresión de los archivos .gz como una tarea puramente de Entrada/Salida (I/O-bound). Se pasó por alto que abrir un flujo comprimido exige un esfuerzo matemático constante por parte del procesador y que, por ende, es una tarea pesada de carga computacional (*CPU-bound*). Además,

tuvimos una pequeña confusión con las herramientas al proponer la interfaz `concurrent.futures` como si fuera un motor de agregación temporal independiente, cuando en realidad es solo un módulo de alto nivel para gestionar hilos o procesos.

Por otro lado, nuestra segunda propuesta aportó la visión estructural: diseñar una arquitectura basada en tuberías (*Pipelining*) híbridas y dividir el archivo masivo en bloques (*chunks*) para asegurar un balance dinámico de la carga en memoria. También describió la dinámica del entorno de Python frente al almacenamiento físico, argumentando con propiedad que los hilos ceden el control del procesador mientras aguardan los datos del disco, lo que evita que el pipeline se penalice durante las fases de Entrada/Salida pura. Sin embargo, en este enfoque no detallamos cómo íbamos a consolidar las agregaciones locales de cada fragmento sin caer en condiciones de carrera o en el uso de bloqueos que pudieran ralentizar el código.

La síntesis grupal nos enseñó que no existe una herramienta universal en Python: el éxito del pipeline radicó en aislar la lectura de datos de la transformación matemática.

3.3. Discusión Explícita del GIL y Estrategia de Diseño Final

El Bloqueo Global del Intérprete (GIL) es el mecanismo de sincronización nativo de CPython que impide que múltiples hilos ejecuten código de Python de forma simultánea en más de un núcleo físico. En nuestro problema, el GIL invalida el uso de hilos puros para las etapas de descompresión y parseo: como estas operaciones manipulan cadenas de texto e interactúan directamente con el bytecode de Python, una solución basada únicamente en `threading` se serializará por completo, arrojando tiempos de ejecución similares o incluso superiores al flujo secuencial básico debido al costo de alternancia de contextos.

Nuestra solución final implementa una Estrategia Híbrida de Flujo por Bloques (*Streaming Chunks*) orientada a evadir el GIL y optimizar el uso de los recursos del sistema:

1. **Particionamiento Dinámico Extremo:** En lugar de cargar el archivo de 2 GB en memoria o dividirlo físicamente en discos (lo cual alteraría las condiciones del benchmark), el script realiza una lectura por flujo continuo. El hilo principal extrae bloques de texto crudo comprimido de tamaño fijo (100.000 líneas por *chunk*).
2. **Mitigación del Overhead de Serialización:** Al comunicar procesos independientes, Python se ve obligado a serializar los objetos mediante la

biblioteca pickle, transmitiendo los bytes a través de canales de Comunicación Interprocesos (IPC). Si enviáramos fila por fila, el costo transaccional de IPC destruiría la eficiencia. Nuestra estrategia mitiga este impacto al empaquetar bloques masivos de texto binario crudo; el costo de serialización se paga una sola vez por cada 100.000 registros.

3. **Agregación Local y Reducción sin Inconsistencias:** Para erradicar el uso de cerrojos concurrentes mutuos que ralentizan el cómputo, cada proceso worker del pool recibe un bloque y computa un diccionario privado de acumulación utilizando estructuras defaultdict. La agregación local calcula las sumas de latencia y frecuencias organizadas por la llave compuesta (ventana_temporal, user_id, endpoint). Al finalizar, cada worker retorna únicamente un resumen consolidado de tamaño ultra-reducido. El proceso padre recolecta estos sub-diccionarios y ejecuta la reducción final de forma secuencial y limpia en la memoria principal, garantizando la consistencia de los datos.

3.4. Pruebas Experimentales y Resultados de Benchmark

Para garantizar la reproducibilidad científica del experimento y evitar sesgos metodológicos, el benchmark se diseñó de forma automatizada. Se ejecutaron ciclos previos de calentamiento térmico (*warm-up*) para estabilizar la tasa de lectura del sistema de archivos y se controló rigurosamente que todas las escalas de workers procesaran exactamente el mismo set de datos sintéticos comprimidos.

Tabla de Rendimiento de Infraestructura en Python

Configuración de Workers (p)	Tiempo de Ejecución (T_p) [s]	Speedup Obtenido (S_p)	Eficiencia Muestral (E_p)
1 Worker (Base)	80,60510s	1,00000x	100,00%
2 Workers	52,95323s	1,52219x	76,11%
4 Workers	28,67715s	2,81078x	70,27%
8 Workers	21,09000s	3,82196x	47,77%

Interpretación de Métricas y Análisis de Escalabilidad

Los resultados demuestran de forma empírica que la estrategia híbrida logra acelerar con éxito el pipeline de procesamiento, reduciendo el tiempo operativo de 80,60 segundos a un mínimo de 21,09 segundos utilizando 8 procesos en paralelo. La curva de *Speedup* muestra una progresión saludable hasta los 4 workers 2,81x, manteniendo una eficiencia del 70,27%. Este rendimiento confirma que el particionamiento masivo por bloques logró transferir la carga pesada del parseo de texto a los diferentes núcleos del procesador de manera equilibrada.

Sin embargo, al configurar la escala máxima de 8 workers, el pipeline exhibe los efectos de la ley de rendimientos decrecientes: el *Speedup* se estabiliza en 3,82x y la eficiencia sufre una contracción, cayendo al 47,77%. Este comportamiento es radicalmente distinto al comportamiento puramente matemático del Ejercicio 1 en C++ y se justifica técnicamente por dos factores estructurales de la arquitectura de Python:

- **La Fracción Secuencial de la Tubería:** Conforme a la Ley de Amdahl, el rendimiento global está topado por la etapa que no se puede ejecutar en paralelo. En este pipeline, la lectura secuencial del archivo comprimido utilizando la biblioteca nativa gzip y la reducción final de los diccionarios maestros en el proceso padre actúan como la restricción lineal del algoritmo.
- **Saturación por Comunicación Interprocesos (IPC):** Al elevar el número de procesos a 8, el costo administrativo de levantar los intérpretes de Python, sumado al tiempo invertido por el sistema operativo en serializar, transmitir por tuberías IPC y deserializar (*unpickling*) los resultados devueltos por cada worker, empieza a competir directamente con el tiempo de cómputo neto del parseo, limitando la escalabilidad lineal ideal del hardware.

Al finalizar el ciclo de evaluación, el pipeline automatizado exportó con éxito el producto de datos consolidado al archivo físico `metricas_ventanas_15min.csv`, cumpliendo de manera íntegra con las especificaciones de persistencia y la métrica funcional de agregación por bloques temporales de 15 minutos.

4. Ejercicio 3: Cálculo de similitud por bloques para embeddings en C++/OpenMP

4.1. Estimación de Memoria y Diagnóstico del Problema

Para dimensionar el desafío de este algoritmo, el primer paso obligatorio fue calcular cuánta memoria RAM se consumiría si intentáramos resolver el problema por fuerza bruta, es decir, construyendo la matriz completa de similitudes para nuestros $N = 20.000$ vectores utilizando precisión doble (double, de 8 bytes):

$$\text{Memoria} = N \times N \times 8 \text{ bytes} = 20.000 \times 20.000 \times 8 = 3.200.000.000 \text{ bytes} \approx 3,2 \text{ GB}$$

Aunque destinar 3,2 GB es perfectamente viable en cualquier computador de escritorio actual, este enfoque presenta un problema crítico de escalabilidad. Si nuestro conjunto de datos creciera a un tamaño mediano dentro de los estándares de la ciencia de datos (por ejemplo, 200.000 vectores), el consumo escalar de la matriz se dispararía de forma cuadrática hasta alcanzar los 320 GB, colapsando de inmediato la memoria de los nodos de cómputo estándar.

Además, desde una perspectiva metodológica, el 99,95% de esa matriz representa información completamente irrelevante para el objetivo del problema si se exige almacenar exclusivamente los 10 vecinos más cercanos por cada observación. Mantener miles de millones de flotantes en la RAM para luego descartar la inmensa mayoría constituye un desperdicio severo de recursos. A esto se suma el costo computacional intrínseco del algoritmo de comparación exhaustiva de todos contra todos, cuya complejidad temporal está gobernada por un orden de magnitud de $O(n^2 \cdot d)$, donde $\text{dimension} = 128$.

4.2. Contraste Crítico de las Propuestas Iniciales y Lecciones Aprendidas

Al igual que en las etapas anteriores, el diseño de la infraestructura paralela definitiva se construyó sobre un debate técnico en el que evaluamos nuestras posturas iniciales para erradicar cualquier error de implementación.

En nuestra primera propuesta, tuvimos la precaución de exigir que los bloques de datos (*chunks*) se mantuvieran en un tamaño acotado para asegurar que cupieran dentro de los niveles de la memoria caché. También acertamos al proponer la directiva `schedule(dynamic)`, intuyendo que el volumen de comparaciones podía variar en el tiempo. Sin embargo, caímos en un error conceptual severo al sugerir la cláusula `reduction` de OpenMP para acumular todos los productos punto antes de buscar los 10 vecinos. En la programación paralela, una reducción combina valores

en variables escalares mediante un operador matemático básico; no está diseñada para concatenar o recolectar colecciones masivas de estructuras vectoriales distribuidas, además de que omitimos realizar la estimación matemática de memoria.

Por otra parte, nuestra segunda alternativa planteó un bucle directo mediante `parallel for` combinando fragmentos para obtener un balance dinámico. Sin embargo, este enfoque sugería abiertamente reducir los resultados consolidándolos en una matriz completa al final del script. Esta propuesta violaba directamente la restricción de diseño del enunciado ("*la matriz completa de similitudes no puede almacenarse en memoria*"), simplificando el problema a un doble bucle plano de fuerza bruta y dejando de lado el tratamiento estructural de la memoria y la lógica de filtrado del *Top-K*.

La puesta en común del equipo nos demostró que la verdadera dificultad no radicaba en paralelizar el doble bucle, sino en diseñar un mecanismo capaz de calcular y descartar candidatos sobre la marcha sin tocar el almacenamiento secundario.

4.3. Estrategia por Bloques, Mantención del Top-10 e Impacto en la Caché

Para cumplir estrictamente con la prohibición de almacenar la matriz global, desarrollamos una Estrategia Exacta basada en el procesamiento en tiempo real (*in-place*) utilizando una estructura de datos de Montículo Máximo (*Max-Heap*) de capacidad fija $K = 10$ por cada fila de vector.

En lugar de poblar una matriz, asociamos a cada vector origen una cola de prioridad privada (`std::priority_queue`). Cuando un hilo de OpenMP calcula el producto punto entre el vector i y el vector j , el valor flotante resultante se evalúa de forma inmediata contra el elemento que reside en la raíz del montículo local:

- Si la similitud calculada es menor que la raíz actual del Heap (que representa el peor valor guardado dentro del grupo de los 10 mejores), el registro se descarta en el acto, liberando los registros del procesador.
- Si el producto punto es mayor, se expulsa la raíz vieja y se inserta el nuevo vecino, reestructurando internamente el montículo en un tiempo óptimo de $O(\log K)$.

Este diseño altera radicalmente el mapa de consumo de hardware. Almacenar la matriz base de embeddings de entrada requiere apenas $20.000 \times 128 \times 8$ bytes $\approx 20,48$ MB. Por su parte, la estructura consolidada de salida para el reporte final del

Top-10 consume $20.000 \times 10 \times (8 \text{ bytes de similitu} + 4 \text{ bytes de ID}) \approx 2,4 \text{ MB}$. Gracias a esta reducción drástica, el espacio de trabajo en memoria RAM pasó de los 3,2 GB teóricos a menos de 23 MB totales. Toda la estructura de datos crítica del problema es capaz de alojarse de forma simultánea dentro de la memoria caché L3 del procesador, eliminando las penalizaciones por transferencia externa.

4.4. Discusión de Sincronización y Balanceo de Carga Triangular

Debido a que los 20.000 vectores de entrada se encuentran normalizados a norma 1, el cálculo del producto punto es directamente equivalente a la similitud de coseno, lo que nos permite aprovechar la propiedad conmutativa de los datos ($X_i \cdot X_j = X_j \cdot X_i$). Esta simetría geométrica implica que el algoritmo solo necesita calcular los elementos de la matriz triangular superior, reduciendo el volumen de operaciones matemáticas exactamente a la mitad.

No obstante, la geometría triangular introduce un desbalance de carga crítico si se distribuye el trabajo de manera convencional. En las primeras iteraciones del bucle externo (cuando i se acerca a 0), el hilo asignado debe realizar casi 20.000 productos punto; en contraste, en las últimas iteraciones (cuando i se aproxima a 19.999), la carga de comparaciones pendientes cae prácticamente a cero.

Para resolver este comportamiento de forma eficiente, justificamos la implementación de la directiva `#pragma omp for schedule(dynamic, 64)`. Al configurar una planificación dinámica con bloques de 64 filas, el entorno de OpenMP crea una cola de tareas en tiempo real. Los hilos toman un paquete de filas, lo procesan, y en cuanto se liberan, solicitan un nuevo bloque al planificador de forma autónoma. Esto evita que los hilos que reciben las últimas filas del problema queden inactivos esperando a los hilos sobrecargados del inicio del set.

Finalmente, el diseño erradicó la necesidad de incluir regiones de exclusión mutua (critical) o cerrojos dentro del cómputo. Como cada hilo opera sobre filas independientes y escribe de manera exclusiva en sus propias colas de prioridad privadas, no existen colisiones de memoria ni condiciones de carrera, garantizando un flujo paralelo libre de bloqueos de sincronización.

4.5. Pruebas Experimentales y Resultados de Benchmark

El ciclo de evaluación experimental se ejecutó de forma automatizada sobre el archivo compilado nativo, registrando de forma autónoma las métricas de velocidad y volcando una muestra funcional de las distancias calculadas.

Métricas de Infraestructura Paralela

Cantidad de Hilos (p)	Tiempo de Ejecución (T_p)[s]	Speedup Obtenido (S_p)	Eficiencia Muestral (E_p)
1 (secuencial)	415,75166s	1,00000x	100,00000%
2	273,95060s	1,51762x	75,88077%
3	144,80977s	2,87102x	71,77549%
4	83,41285s	4,98426x	62,30330%

Interpretación de Métricas y Escalabilidad

A diferencia de las experiencias previas, este ejercicio se comporta como un problema estrictamente limitado por la capacidad de procesamiento del hardware (CPU-Bound), debido a que la cantidad de instrucciones lógicas y comparaciones flotantes es masiva frente al tamaño total del archivo en disco ($O(n^2 \cdot d)$). El pipeline logró acelerar el cómputo desde una base secuencial de 415,75 segundos (6 minutos y 55 segundos) hasta un mínimo optimizado de 83,41 segundos al activar los 8 hilos.

La curva de *Speedup* muestra una aceleración ascendente y constante, alcanzando un factor de 4,98x con la máxima capacidad de hilos. Por su parte, la eficiencia experimenta un descenso escalonado, estabilizándose en un 62,30% al llegar a los 8 trabajadores. Esta pérdida del 37% de rendimiento lineal se fundamenta en la naturaleza del costo algorítmico de la estructura de datos elegida: ordenar el *Max-Heap* local e insertar elementos en la cola de prioridad añade un componente de control no lineal que introduce bifurcaciones lógicas dentro de los núcleos de la CPU, limitando el aprovechamiento asintótico ideal que describe la Ley de Amdahl.

Muestra de Resultados Comprobatorios (Top-3 Vecinos)

Para certificar la fidelidad y precisión del procesamiento por bloques, el programa imprimió los índices y distancias resultantes para los primeros vectores del conjunto:

Vector Origen	Vecino Cercano 1 (ID/Sim)	Vecino Cercano 2 (ID/Sim)	Vecino Cercano 3 (ID/Sim)
Vector 0	ID: 12191 (0,3414)	ID: 13217 (0,3241)	ID: 16923 (0,3229)
Vector 1	ID: 17011 (0,3401)	ID: 14581 (0,3351)	ID: 9944 (0,3340)
...	...	...	...
Vector 8	ID: 10788 (0,3335)	ID: 1450 (0,3298)	ID: 12679 (0,3270)
Vector 9	ID: 11643 (0,3339)	ID: 10685 (0,3244)	ID: 9721 (0,3063)

Métrica de Calidad: Dado que la estrategia implementada recorre y evalúa de manera exhaustiva el espacio completo de vecindad de la matriz, la solución garantiza un 100% de precisión (métrica Recall@10 = 1,0).

Los resultados impresos comprueban la consistencia del filtro analítico: las similitudes calculadas se ubican de forma homogénea en un rango de dispersión coherente (entre 0,30 y 0,37), y los identificadores de los vecinos varían dinámicamente entre las 20.000 observaciones posibles. Esto demuestra que la técnica del montículo local por bloques logró discriminar con total exactitud matemática a los elementos más cercanos de la muestra, resolviendo el problema espacial y temporal con el más alto estándar de optimización de código.

5.- Discusión transversal: Trade-offs de diseño en algoritmos de alto rendimiento

El desarrollo de este trabajo nos demostró que la optimización a gran escala no responde de una única manera. Por el contrario, cada escenario nos obligó a equilibrar de manera crítica el paralelismo, la comunicación, la memoria, la sincronización y la complejidad algorítmica.

5.1. Paralelismo vs. Sincronización

Uno de los aprendizajes más claros fue que añadir unidades de procesamiento es inútil si la sincronización bloquea los núcleos. En el Ejercicio 1, el volumen de 50 millones de filas nos tentó a realizar acumulaciones continuas. Sin embargo,

introducir una directiva `#pragma omp critical` en el bucle interno habría forzado al hardware a gestionar 800 millones de cerrojos mutuos, haciendo el código paralelo más lento que el secuencial. Lo resolvimos privatizando los acumuladores en buffers locales y reduciendo la sincronización crítica a una sola vez por hilo al final de la fase. En el Ejercicio 3 llevamos esto al extremo ideal: al diseñar colas de prioridad locales por fila, eliminamos por completo los bloqueos durante el cálculo, permitiendo que cada hilo operara de forma independiente y libre de fricciones.

5.2. Comunicación vs. Complejidad

El contraste entre C++ y Python dejó en evidencia cómo el modelo de ejecución condiciona el rendimiento mediante el *overhead* de comunicación. Con OpenMP operamos bajo un paradigma de memoria compartida; los hilos acceden a punteros contiguos en la RAM sin necesidad de copiarse datos, reduciendo el costo de comunicación a cero.

Con multiprocessing en Python, la realidad fue opuesta. Para evadir el Bloqueo Global del Intérprete (GIL), nos vimos obligados a levantar procesos aislados que no comparten memoria. Esto obligó al sistema a serializar cada fragmento mediante pickle y transmitirlo por tuberías IPC, un costo transaccional que habría destruido la eficiencia si se hacía fila por fila. Tuvimos que elevar la complejidad lógica implementando un flujo continuo por bloques (*streaming chunks*) de 100.000 líneas, pagando el costo de la comunicación una sola vez por paquete y encontrando un equilibrio entre la granularidad y la mensajería.

5.3. Memoria vs. Cómputo

La cantidad de memoria modificó radicalmente nuestros benchmarks, permitiéndonos clasificar las cargas según su limitación física. El Ejercicio 1 se consolidó como un problema limitado por memoria (*Memory-Bound*), donde mover 25 GB de datos para realizar operaciones aritméticas elementales terminó saturando el ancho de banda de la placa madre. Al subir a 8 hilos, la CPU procesó más rápido de lo que el bus de datos podía transferir, forzando ciclos de espera inactivos que limitaron la escalabilidad según la Ley de Amdahl.

Por el contrario, el Ejercicio 3 se comportó como un problema limitado por cómputo (*CPU-Bound*). A pesar de su costo cuadrático ($O(n^2 \cdot d)$), nuestra estrategia de filtrado *in-place* redujo el espacio de trabajo en RAM de los 3,2 GB teóricos a menos de 23 MB totales. Al comprimir las estructuras críticas mediante un montículo local, logramos que el problema residiera por completo dentro de la memoria caché L3 del procesador. Al no tener que salir a la RAM externa, los hilos calcularon a la máxima velocidad del silicio, alcanzando una aceleración sólida de 4,98x.

El rendimiento no se consigue simplemente activando hilos. Un verdadero diseño en Ciencia de Datos requiere diagnosticar primero la naturaleza de la carga para seleccionar la estrategia que minimice la sincronización costosa y mantenga los datos lo más cerca posible de los núcleos de ejecución.

6.- Conclusiones

El desarrollo de este trabajo nos demostró que el rendimiento en Ciencia de Datos no se consigue simplemente activando hilos de forma arbitraria. Un verdadero diseño de alto rendimiento requiere entender primero la naturaleza física de la carga para seleccionar la estrategia de software que minimice la sincronización costosa y maximice la cercanía de los datos a los núcleos de ejecución.

A lo largo de las experiencias, el principal aprendizaje técnico fue la necesidad de diagnosticar y aislar las variables para proteger las memorias caché del procesador. En C++ aprendimos que repartir iteraciones a ciegas destruye el paralelismo si múltiples hilos escriben de forma continua en posiciones contiguas, por lo que diseñar buffers locales privados y estructuras de prioridad independientes por fila fue lo que nos permitió neutralizar el cuello de botella de la sección crítica. Por su parte, la experiencia en Python nos enseñó a convivir con las restricciones del bloqueo global del intérprete (GIL) mediante una arquitectura híbrida, donde dividimos las tareas de forma estratégica, usando hilos para gestionar las esperas físicas de disco y delegando el parseo pesado a procesos en núcleos aislados de forma real.

A pesar de estas optimizaciones, cada solución evidenció límites estructurales insalvables. La normalización masiva se consolidó como un escenario limitado por memoria (*Memory-Bound*), donde la CPU procesó las sumas planas tan rápido que terminó saturando el ancho de banda del bus de datos de la placa madre, forzando ciclos de espera inactivos bajo la Ley de Amdahl. En el pipeline de logs la limitación estuvo impuesta por el costo administrativo de la comunicación interprocesos (IPC), donde el tiempo dedicado a serializar y transmitir objetos mediante *pickle* empezó a competir con el cómputo neto. Finalmente, el cálculo de embeddings demostró que, aunque comprimimos las estructuras críticas para que residieran en la caché L3, el costo de ordenamiento del montículo local ($O(\log K)$) bajo carga cuadrática representó la restricción final del hardware.

Estas decisiones arquitectónicas cambiarían radicalmente si se modificaran las condiciones iniciales del problema. Si el tamaño del conjunto de datos escalara exponencialmente a miles de millones de registros, el paradigma de memoria compartida en un solo nodo colapsaría, obligándonos a mutar hacia sistemas de memoria distribuida utilizando MPI o clústeres en Apache Spark. Asimismo, si el

problema de embeddings priorizara la velocidad en milisegundos sobre la exactitud matemática, descartaríamos el barrido exhaustivo de todos contra todos para implementar algoritmos de búsqueda aproximada (ANN) basados en grafos HNSW o árboles KD. Por último, ante la disponibilidad de hardware dedicado, tanto la normalización como los productos punto de los vectores se delegarían por completo a la computación masivamente paralela en tarjetas gráficas (GPU) mediante CUDA, transformando algoritmos de minutos en ejecuciones de fracciones de segundo.

7.- Referencias y anexos: Fuentes, comandos, código y evidencia experimental

7.1. Registro de Infraestructura y Configuración de Entorno

Para garantizar la reproducibilidad científica de las estimaciones y benchmarks presentados en este informe, se explicita el entorno físico y lógico utilizado en las pruebas:

- **Sistema Operativo:** Windows 11 Home
- **Arquitectura de CPU:** AMD Ryzen 5 5500 (6 núcleos físicos, 12 hilos lógicos disponibles).
- **Memoria RAM:** 16 GB DDR4.
- **Versión de Python:** Python 3.12.3 (64-bit).
- **Compilador C/C++:** GCC g++ versión 13.2.0 (distribución MinGW-w64).
- **Soporte OpenMP:** OpenMP 4.5 / 5.0 integrado nativamente en el compilador GCC.

7.2. Parámetros Experimentales y Métricas de Datos

- **Ejercicio 1 (Normalización Masiva):** * *Tamaño de datos:* Matriz sintética contigua de $50.000.000 \times 16$ elementos de punto flotante de doble precisión (double), equivalente a un volumen bruto de 6,4 GB de datos en memoria para una sola matriz (aproximadamente 25 GB movilizados entre buffers intermedios y de salida), simulando un 5% de valores nulos (NaN).
 - Hilos evaluados (OMP_NUM_THREADS): 1, 2, 4 y 8 hilos hilos en ejecución.
 - Repeticiones: Cada prueba de hilos se ejecutó 5 veces de forma consecutiva, registrando el promedio de tiempo de CPU neto.
- **Ejercicio 2 (Pipeline de Logs):**

- Tamaño de datos: Archivos masivos comprimidos en formato .gz conteniendo 5.000.000 de registros de texto planos distribuidos por líneas.
- Configuración de procesos: 1, 2, 4 y 8 trabajadores paralelos independientes (max_workers).
- Repeticiones: 3 ejecuciones completas por configuración de procesos para estabilizar variaciones de I/O de disco.
- **Ejercicio 3 (Vecinos Cercanos):**
 - Tamaño de datos: Matriz de 20.000 embeddings densos de dimensión fija 128, simulando una matriz completa de correlaciones exhaustivas de 400.000.000 de emparejamientos en punto flotante.
 - Hilos evaluados: 1, 2, 4 y 8 hilos ejecutados de forma aislada.
 - Repeticiones: 5 ejecuciones automatizadas por configuración.

7.3. Comandos de Compilación y Ejecución

Códigos en C++ (OpenMP - Ejercicios 1 y 3)

La compilación se gestiona de forma centralizada mediante un archivo Makefile configurado con optimizaciones de optimización agresivas a nivel de registros (-O3) y el enlace de la librería paralela:

None

```
# Comando de compilación automatizado
```

```
make
```

```
# Comandos de compilación manual alternativos
```

```
g++ -O3 -fopenmp normalizacion.cpp -o normalizacion
```

```
g++ -O3 -fopenmp ejercicio3.cpp -o ejercicio3
```

```
# Comandos de ejecución física de los ejecutables binarios
```

```
./normalizacion
```

```
./ejercicio3
```

Códigos en Python (Multiprocessing - Ejercicio 2)

El script de procesamiento por bloques no requiere la descarga de librerías externas en el entorno de ejecución, llamando directamente al intérprete optimizado:

C/C++

```
# Comando opcional para la generación del dataset de accesos
```

```
python generar_logs.py
```

```
# Comando de ejecución del benchmark del pipeline distribuido
```

```
python pipeline_logs.py
```

7.4. Repositorio de Código Fuente y Control de Versiones

El ecosistema de desarrollo completo, incluyendo los códigos fuentes estructurados, scripts de prueba, archivos Makefile de automatización y logs de salida, se encuentra disponible en el repositorio público de GitHub:

- **Enlace al repositorio institucional:**
https://github.com/cperezfl/parte_2_grupal_christian_perez_benjamin_zamora

7.5. Declaración de Uso de Herramientas de Inteligencia Artificial

En concordancia las condiciones respecto a la honestidad académica y uso declarado de tecnologías emergentes, el grupo detalla el alcance del uso de IA:

- **Herramienta utilizada:** Gemini (Google).
- **Alcance del uso:** Se implementó explícitamente como un asistente técnico de soporte para acelerar el diseño de la estructura base de los códigos fuente, la resolución de errores de sintaxis en el manejo de punteros, la generación del archivo estandarizado Makefile para OpenMP y el refinamiento de estilo gramatical/ortográfico del informe técnico.
- **Criterio y Autoría Humana:** Ningún fragmento conceptual, análisis arquitectónico de hardware ni conclusión de este informe fue copiado de

forma automatizada o acrítica. Las justificaciones de los fenómenos observados (como el *Memory Wall*, los cuellos de botella por IPC o el costo del ordenamiento del Max-Heap), la integración lógica del código y la revisión matemática fueron validadas y desarrolladas en su totalidad por los integrantes del grupo.

7.6. Fuentes y Referencias Consultadas

1. Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003). *Introduction to Parallel Computing* (2nd ed.). Addison-Wesley.
2. Pacheco, P. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann.
3. Hager, G., & Wellein, G. (2010). *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press.
4. McCool, M., Reinders, J., & Robison, A. (2012). *Structured Parallel Programming*. Morgan Kaufmann.
5. OpenMP Architecture Review Board. (2020). *OpenMP Application Program Interface Specification Version 5.0*. Recuperado de <https://www.openmp.org/>
6. Python Software Foundation. (2026). *Concurrent Execution — concurrent.futures ProcessPoolExecutor Documentation*. Recuperado de <https://docs.python.org/3/>